Contents

1 The Gram-Schmidt Process *3*

1.1 The Process *3*

1.2 Example Calculation *4*

1.3 The Gram-Schmidt Process in Python *6*

1.4 Where to Learn More *7*

2 Singular Value Decomposition *9*

2.1 Definition *9*

2.2 Applications of SVD *9*

2.3 Calculating SVD Manually *10*

2.4 Singular Value Decomposition with Python *13*

2.5 Sign Ambiguity *14*

2.6 SVD Applied to Image Compression *14*

2.7 Where to Learn More *15*

3 Tackling Difficult Problems: Positive Semidefinite Matrices *17*

3.1 NP Problems *17*

3.2 A Past Approach: Minimizing Errors in Neural Networks *18*

3.3 Positive Definite and Semidefinite Matrices *19*

3.4 Identifying and Constructing a Positive Semidefinite Matrix *20*

3.5 The Max Cut Problem *21*

3.6 The Max Cut Problem Solved in Python *23*

4 Introduction to C *25*

4.1 Compiled vs. Interpreted *25*
4.2 Your First C Program *26*
4.3 Types and Variables *26*
4.4 Printing with `printf` *27*
4.5 Conditionals and Loops *28*
4.5.1 if / else *28*
4.5.2 for and while loops *28*
4.6 Functions *29*
4.7 The Stack: Blackboards for Functions *30*
4.7.1 Parameters are copies *31*
4.8 Addresses and Pointers *31*
4.8.1 Reading and writing through a pointer *32*
4.8.2 NULL pointers *32*
4.9 Pass-by-Reference *33*
4.10 Structs: Grouping Related Data *34*
4.11 The Heap *35*
4.11.1 The arrow operator *36*
4.11.2 Releasing heap memory *36*
4.11.3 A complete heap example *37*
4.11.4 Heap arrays *37*
4.12 What Python Objects Actually Are *38*
4.13 Exercises *38*

A Answers to Exercises ***41***

Index 43

Chapter 1

The Gram-Schmidt Process

The Gram-Schmidt Process is a method use to transform a set of linearly independent vectors to a set of orthogonal (perpendicular) vectors. The original vectors and the transformed vectors span the same subspace.

The process was named after two mathematicians: Jørgen Pedersen Gram, a Danish actuary mathematician, and Erhard Schmidt, a German mathematician. The men developed the orthogonalization process independently. Gram introduced the process in 1883, whereas Schmidt did his work in 1907. It was not named the Gram-Schmidt process until sometime later, after both mathematicians became well known in the mathematical community.

In the last chapter, you learned how to calculate a projection of one vector on another. Given two vectors, u and v, the projection separates u into the part that is orthogonal to v and the part that has a dependency with v. The Gram-Schmidt Process builds on the notion of projections to iteritively strip away dependencies between vectors until the vectors that remain are orthogonal. If there happens to be a vector that is a linear combination of the others, that vector will be reduced to the zero vector. The vectors that remain define a new basis for space spanned by the original set of vectors.

Gram-Schmidt has many practical applications in science and engineering, such as:

1. In signal processing, it can represent an audio signal with fewer components making it easier to isolate and remove noise.
2. In statistics and data analysis, it can reduce the complexity of a dataset so that it is easier to see which aspects or features contribute to the analysis.

1.1 The Process

The Gram-Schmidt Process orthonormalizes a set of vectors in an inner product space, most commonly the Euclidean space $\mathbb{R}^n$. The process takes a finite, linearly independent set $S = \{v_1, v_2, \ldots, v_k\}$ for $k \leq n$, and generates an orthogonal set $S' = \{u_1, u_2, \ldots, u_k\}$ that spans the same k-dimensional subspace of $\mathbb{R}^n$ as S.

Let's look at how the process works. Given a set of vectors $S = \{v_1, v_2, \ldots, v_k\}$, the Gram-Schmidt Process is as follows:

1. Let $u_1 = v_1$.
2. For $j = 2, 3, \ldots, k$:
   (a) Let $w_j = v_j - \sum_{i=1}^{j-1} \frac{\langle v_j, u_i \rangle}{\langle u_i, u_i \rangle} u_i$
   (b) Let $u_j = w_j$

Here, $\langle ., . \rangle$ denotes the inner product.

The set of vectors $S' = \{u_1, u_2, \ldots, u_k\}$ obtained from this process is orthogonal, but not necessarily orthonormal. To create an orthonormal set, you simply need to normalize each vector u_i to unit length. That is, $u_i' = \frac{u_i}{\|u_i\|}$, where $\|.\|$ denotes the norm (or length) of a vector.

Among other things, making vectors orthonormal simplifies calculations makes it easier to define rotations and transformations, and provides a framework for calculations in fields such as quantum mechanics.

1.2 Example Calculation

Given a set of linearly independent vectors, we will use the Gram-Schmidt Process to find an orthogonal basis.

Let

$$W = Span(x_1, x_2, x_3)$$

where

$$x_1 = (1, 2, -2)$$
$$x_2 = (1, 0, -4)$$
$$x_3 = (5, 2, 0)$$

The three orthogonal vectors will define the same subspace as the original vectors.

The first vector of the orthogonal subspace is easy to define. We set it to be the same as x_1.

$$v_1 = x_1 = (1, 2, -2)$$

The second orthogonal vector is a projection of x_2 onto v_1. You learned projections in the last chapter, so this should be fairly straightforward.

$$v_2 = x_2 - \frac{x_2 v_1}{v_1 v_1} v_1$$

Substitute the values:

$$v_2 = (1, 0, -4) - \frac{(1, 0, -4)(1, 2, -2)}{(1, 2, -2)(1, 2, -2)} (1, 2, -2)$$

Calculate the coefficient for v_1:

$$v_2 = (1, 0, -4) - \frac{9}{9}(1, 2, -2)$$

Perform the subtraction:

$$v_2 = (0, -2, -2)$$

The third vector for the orthogonal subspace is a projection onto v_1 and v_2.

$$v_3 = x_3 - \frac{x_3 v_1}{v_1 v_1} v_1 - \frac{x_3 v_2}{v_2 v_2} v_2$$

Substitute the values:

$$v_3 = (5, 2, 0) - \frac{(5, 2, 0)(1, 2, -2)}{(1, 2, -2)(1, 2, -2)}(1, 2, -2) - \frac{(5, 2, 0)(1, 0, -4)}{(1, 0, -4)(1, 0, -4)}(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (9/9)(1, 2, -2) - (-4/8)(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (1, 2, -2) + (1/2)(1, 0, -4)$$

$$v_3 = (5, 2, 0) - (1, 2, -2) + (0, -1, -1)$$

$$v_3 = (4, -1, 1)$$

This set of vectors is orthogonal, so we need to normalize them so that the vectors are orthonormal. Recall that an orthonormal vector has a length of 1 and is computed using this formula:

$$normalizedVector = vector / np.sqrt(np.sum(vector * *2))$$

Thus the normalized set of vectors is:

$$v_1 = (0.33, 0.67, -0.67)$$

$$v_2 = (0.0, -0.71, -0.71)$$

$$v_3 = (0.94, -0.24, 0.24)$$

Exercise 1 Gram-Schmidt Process

Use the Gram-Schmidt Process to find an orthogonal basis for the span defined by x_1, x_2 where:

$$x_1 = (1, 1, 1)$$

$$x_2 = (0, 1, 1)$$

Working Space

Answer on Page 41

1.3 The Gram-Schmidt Process in Python

Create a file called `vectors_gram-schmidt.py` and enter this code:

```
# import numpy to perform operations on vector
import numpy as np

# Find an orthogonal basis for the span of these three vectors
x1 = np.array([1, 2, -2])
x2 = np.array([1, 0, -4])
x3 = np.array([5, 2, 0])

# v1 = x1
v1 = x1
print("v1 = ",v1)

# v2 = x2 - (the projection of x2 on v1)
v2 = x2 - (np.dot(x2,v1)/np.dot(v1,v1))*v1
print("v2 = ", v2)

# v3 = x3 - (the projection of x3 on v1) - (the projection of x3 on v3)
v3 = x3 - (np.dot(x3,v1)/np.dot(v1,v1))*v1 - (np.dot(x3,v2)/np.dot(v2,v2))*v2
print("v3 =", v3)

# Next, normalize each vector to get a set of vectors that is both orthogonal and
↪  orthonormal:
v1_norm = v1 / np.sqrt(np.sum(v1**2))
v2_norm = v2 / np.sqrt(np.sum(v2**2))
v3_norm = v3 / np.sqrt(np.sum(v3**2))
print("v1_norm = ", v1_norm)
print("v2_norm = ", v2_norm)
```

```
27 print("v3_norm = ", v3_norm)
```

1.4 Where to Learn More

Watch this video from Khan Academy about the Gram-Schmidt Process: https://www.youtube.com/watch?v=rHonltF77zI

Chapter 2

Singular Value Decomposition

In the previous chapter, you learned how to calculate eigenvalues and eigenvectors. However, not every matrix has them. For those matrices, singular values and singular vectors are analogous features.

Singular Value Decomposition (SVD) is a matrix factorization technique that breaks down a matrix into three matrices that represent the structure and properties of the original matrix. The decomposed matrices make calculations easier and provide insight into the original matrix. Essentially, SVD can transform a high dimension, highly variable set of data into a set of uncorrelated data points that reveal subgroupings that you might not have noticed in the original data. SVD tells us that a linear transformation can be thought of as a rotation, scaling, and another rotation.

2.1 Definition

For any $m \times n$ matrix A, SVD decomposes the matrix into three matrices.

$$A = U\Sigma V^T \tag{2.1}$$

- U is an orthogonal matrix whose size is $m \times m$. Its columns are the eigenvectors of AA^T. These are the left singular vectors of A. Because U is orthogonal, $U^TU = I$.
- V is an orthogonal matrix whose size is $n \times n$ matrix. Its columns are the eigenvectors of A^TA. These are the right singular vectors of A. Because V is orthogonal, $V^TV = I$.
- Σ is a diagonal matrix that is the same size as A. Its diagonal contains the singular values of A, arranged in descending order. These values are the square roots of the eigenvalues of both A^TA and AA^T.

2.2 Applications of SVD

SVD has numerous applications:

- It is used in machine learning and data science to perform dimensionality reduction, particularly through a technique known as Principal Component Analysis (PCA).

- In numerical linear algebra, SVD is used to solve linear equations and compute matrix inverses in a more numerically stable way.
- It is used in image compression, where low-rank approximations of an image matrix provide a compressed version of the original image.

2.3 Calculating SVD Manually

You might be inclined to skip this example because the computations are lengthy. Why would anyone do this when they can use a computing language, like Python, to calculate the SVD with essentially one command? We show this so you can understand what goes on "under the hood" when you compute SVD programmatically.

After you read through this example, you will see how to use Python to compute SVD. Next, you will see an example of using SVD for image compression. Finally, you will be given an exercise to compute the SVD. For this, you will need to write your own Python script.

Let's find the SVD for matrix A. Recall that we want to find U. Σ, and V^T such that:

$$A = U\Sigma V^T \tag{2.2}$$

$$A = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$$

$$U = AA^T$$

Calculating

$$AA^T$$

will give us a square matrix:

$$A^T = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix}$$

$$AA^T = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix}$$

Next, we will find the eigenvalues and eigenvectors of A^T. This is a chance to apply what you learned in the previous chapter. We know that:

$$Av = \lambda v \tag{2.3}$$

So:

$$\begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Rewrite as a set of equations:

$$11x_1 + x_2 = \lambda x_1$$

$$x_1 + 11x_2 = \lambda x_2$$

Then rearrange:

$$(11-\lambda)x_1 + x_2 = 0$$

$$x_1 + (11-\lambda)x_2 = 0$$

Solve for λ:

$$\begin{bmatrix} (11-\lambda), 1 \\ 1, (11-\lambda) \end{bmatrix} = 0$$

And as equations:

$$(11-\lambda)(11-\lambda) - 1 \cdot 1 = 0$$

$$(\lambda-10)(\lambda-12) = 0$$

These are the eigenvalues.

$$\lambda = 10$$

$$\lambda = 12$$

When substituted into the original equations, you get the eigenvectors. For

$$\lambda = 10$$

:

$$(11-10)x_1 + x_2 = 0$$

$$x_1 = -x_2$$

We will set

$$x_1$$

to 1 and get this eigenvector:

$$[1, -1]$$

For

$$\lambda = 10$$

:

$$(11-12)x_1 + x_2 = 0$$

$$x_1 = x_2$$

We will set

$$x_1$$

to 1 and get this eigenvector:

$$[1, 1]$$

The matrix is:

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Next, you need to apply the Gram-Schmidt process to the column vectors. After that, you will have U, the $m \times m$ matrix whose columns are eigenvectors of AA^T. These are the left singular vectors of A. After you apply Gram-Schmidt, you should end up with:

$$U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix}$$

The process for calculating V is the same as the calculation for U, except:

$$V = A^T A$$

$$A^T A = \begin{bmatrix} 3 & -1 \\ 1 & 3 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 10 & 4 \\ 2 & 4 & 2 \end{bmatrix}$$

After applying the process we applied to solve for U, you get:

$$V = \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{5} & 1/\sqrt{30} \\ 2/\sqrt{6} & -1/\sqrt{5} & 2/\sqrt{30} \\ 1/\sqrt{6} & 0 & -5/\sqrt{30} \end{bmatrix}$$

However, you want V_T:

$$V_T = \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

You have only to calculate Σ, a diagonal matrix that is the same size as A. The diagonal contains the singular values of A, arranged in descending order. These are the square roots of the eigenvalues of both $A^T A$ and AA^T.

Because the non-zero eigenvalues of U are the same as V, let's use the eigenvalues we calculate for U, 10 and 12. Note that Σ will not be of the correct dimension to reconstruct the original matrix unless we add a column. By adding a zero column you'll be able to multiply between U and V:

$$\Sigma = \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{12} & 0 \end{bmatrix}$$

You can check your work by multiplying the decomposed matrices. This should return the original matrix.

$$A = U\Sigma V^T$$

$$= U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{bmatrix} \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{12} & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

$$= \begin{bmatrix} \sqrt{12}/\sqrt{2} & \sqrt{10}/\sqrt{2} & 0 \\ \sqrt{12}/\sqrt{2} & -\sqrt{10}/\sqrt{2} & 0 \end{bmatrix} \begin{bmatrix} 1/\sqrt{6} & 2/\sqrt{6} & 1/\sqrt{6} \\ 2/\sqrt{5} & -1/\sqrt{5} & 0 \\ 1/\sqrt{30} & 2/\sqrt{30} & -5/\sqrt{30} \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}$$

2.4 Singular Value Decomposition with Python

Create a file called `vectors_decomposition.py` and enter this code:

```
# Singular-value decomposition
import numpy as np
from numpy import array
from scipy.linalg import svd
from numpy import diag
from numpy import dot
from numpy import zeros

# Define a matrix
A = array([[1, 2], [3, 4], [5, 6]])

print("Matrix (3x2) to be decomposed: ")
print(A)

# CalculateSVD
U, S, VT = svd(A)
print("Matrix (3x3) that represents the left singular values of A:")
print(U)
print("Singular values:")
print(S)
print("Matrix (2x2) that represents the right singular values of A:")
print(VT)

# Check if the decomposition by rebuilding the original matrix
# The singular values must be in an m x n matrix
# Create a zero matrix with the same dimension as A
Sigma = zeros((A.shape[0], A.shape[1]))
# Populate Sigma with n x n diagonal matrix
Sigma[:A.shape[1], :A.shape[1]] = diag(S)
# Reconstruct the original matrix
A_Rebuilt = U.dot(Sigma.dot(VT))
print("Original matrix:")
print(A_Rebuilt)
```

2.5 Sign Ambiguity

You might notice that at times, the absolute values in the U and V^T matrices are correct, but that the signs vary from what you see as the answer. For example, when you compare a manually calculated SVD with one done in Python the signs might not agree. Both decompositions of A are valid. Both decompositions will satisfy:

$$A = U\Sigma V^T$$

Note that the S diagonal values will always be positive.

The sign ambiguity has implications. For example, when using SVD to compress data, if some of the signs are flipped, the data can have artifacts. At this point in your education, you don't need to concern yourself with it, except when you are comparing SVD results for the same matrix.

Exercise 2 Single Value Decomposition

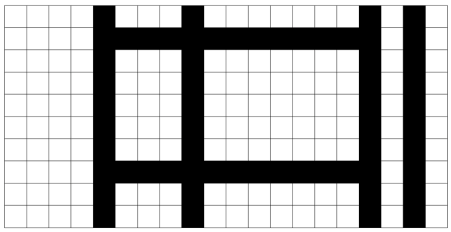
Modify your Python code to calculate SVD for the matrix in the worked out example. Did you arrive at the same answer? Keep in mind that Python will compute square roots and present fractions as decimal. Take a look at the signs for the values in the U and V^T matrices. Are they the same, or is this an example of sign ambiguity?

Working Space

Answer on Page 41

2.6 SVD Applied to Image Compression

This image consists of a grid of 20 by 10 pixels, each of which is either black or white.



It is a simple image that has only two types of columns–ideal for data compression. A row is either the first pattern or the second.

We can represent the data as a 20 by 10 matrix whose 200 entries are either 0 for black or 1 for white.

$$\begin{bmatrix}
00001000100000001010\\
00001111111111111010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001000100000001010\\
00001111111111111010\\
00001000100000001010\\
00001000100000001010
\end{bmatrix}$$

When you perform an SVD on this matrix, there are only two non-zero singular values, 6.79 and 3.72. (You are welcome perform the calculation in Python.) Thus, you can represent the matrix as:

$$A = U_1 S_1 V_1 + U_2 S_2 V_2$$

This means there are two u vectors, each with 20 entries, two v vectors each with 10 entries, and two singular values. Add those up: 2*20 + 2*10 + 2 = 62. This implies that the image can be represented by 62 values instead of 200. If you look back at the image, you can see that there are many dependent columns and very few independent ones.

This is a simple image and a small pixel matrix, but it should give you a sense of how SVD can decompose an image in a way that identifies how much of the image is redundant, and therefore can be compressed.

2.7 Where to Learn More

We Recommend a Singular Value Decomposition. This American Mathematical Society publication focuses the geometry of SVD. What we like about the article is that it shows both

graphically and numerically how SVD can be used for data compression on images and for noise reduction. The data compression example in your workbook is based on this article. https://www.ams.org/publicoutreach/feature-column/fcarc-svd

Sign Ambiguity in Singular Value Decomposition (*SVD*). This is a good article for those who want a deeper understanding of sign ambiguity. https://www.educative.io/blog/sign-ambiguity-in-singular-value-decomposition

Singular Value Decomposition Tutorial. This PDF starts by defining points, space, and vectors and works through all the concepts you need to tackle SVD. It is one of the few resources that has a completely worked out example of manually calculating SVD. The example in this chapter is from that tutorial. If you read the entire paper, you will find that it is a good review of the concepts you have studied in previous chapters. https://rb.gy/j6s0w

Chapter 3

Tackling Difficult Problems: Positive Semidefinite Matrices

With all the computing power available today, you would think no problem would be too difficult to tackle. However, this is not the case. There is a category of problems called NP (nondeterministic polynomial time) whose solution is easy to verify, but whose computation is difficult to perform. This is because there is no straightforward algorithm and any brute-force method would take too much time. For these problems, all we can do is to develop an efficient algorithm that can find a solution in a reasonable amount of time. While the solution might not be the most optimal, the goal is to find the best solution possible in a short time.

As you learn more about optimization techniques, you will come across many efficiency algorithms that have been used throughout the years. In the 1990s, the field of optimization changed with the discovery that algorithms based on semidefinite positive matrices can achieve a higher efficiency than seen in the past. Today, there is an entire field of programming called Semidefinite Programming that is based on the use of semidefinite positive matrices.

First we will take a look at what some of the NP problems are. Next, we will describe the intuition behind semidefinite positive matrices. We will take a look at one NP problem, then introduce the Python module to use for solving these problems.

3.1 NP Problems

In the world of mathematics, easy problems are referred to as P, or polynomial time, problems. In simple terms, this means the problem can be solved quickly, and it is easy to verify that the solution is correct. Addition, subtraction, division, multiplication, square roots, and matrix-vector multiplication are just a few examples. NP problems, as stated in the introduction, can't be computed in a reasonable amount of time, but solutions are usually easy to verify. The game of Sudoku is one such example. It is easy to verify a correct solution, but writing a generalized algorithm to solve any Sudoku game is an NP problem.

NP problems show up in many other situations, such as:

- Designing robust communication networks

- Scheduling tasks without conflicts
- Managing supply chains
- Detecting patterns in biological networks
- Figuring out subgroups in social networks
- Predicting the structure of proteins

For each of these , think of large scale problems for which there are many variables. A cloud computing company that provides AI services to thousands of clients must be able to schedule tasks efficiently and in a timely manner, as well as manage the power needed for the computers and cooling the data center. Supply chain management is crucial to figuring out how to pick up, transport, and distribute goods to help provide disaster relief. Understanding protein structure is important for designing drugs that can tackle specific conditions. All we can do for each of these situations is to find an optimal solution, but we cannot find the definitive solution.

3.2 A Past Approach: Minimizing Errors in Neural Networks

When neural networks were first being developed to recognize things (faces, letters, music, and so on), the goal was to calculate weights between network nodes that would minimize the recognition error. The error space could be visualized as a surface of valleys and hills. The lowest point would have the least error. The idea behind the iterative weight calculations for training the network was to descend down the gradient until reaching the low point. Without getting into the mathematics, you can see by looking at this figure that there are two valleys. Some neural network training resulted in ending at a low point, but not the lowest point. Wouldn't it be great if you could formulate an optimization problem so that you would be guaranteed to land at the lowest point? That is where semidefinite programming can help.

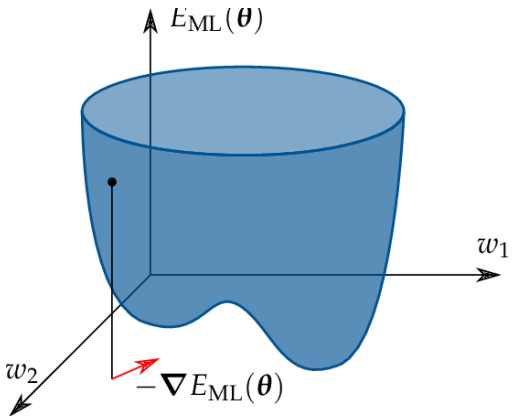
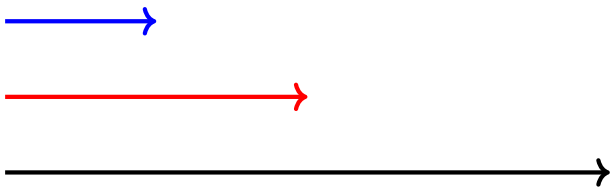


Figure 3.1: A neural network error surface with two valleys. Gradient descent can converge to a local minimum rather than the global minimum.

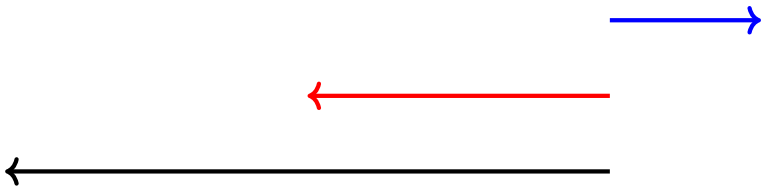
3.3 Positive Definite and Semidefinite Matrices

Unlike matrices that are defined by their content (such as identity matrix, zero matrix, and diagonal matrix), positive definite and semidefinite matrices are characterized by the result they produce. They have an analog in the scalar world, so let's first look at that. Let's take the scalars a and b and treat them as vectors. ab is then the dot product. If $a > 0$, then ab will take on the sign of b. If $a < 0$, then ab will have the opposite sign of b. If you look at this in a graph, you can see that in the first case, ab stays on the same side of the origin, but in the second case, ab flips

For $a = [2], b = [4]$



For $a = [3], b = [-4]$. the result of multiplication flips a to the other side of the graph.



The notion of "staying on the same side" is positive definite. The notion of flipping is negative definite. Positive definite means that $x > 0$, so the result is a positive number. Positive semidefinite means that $x >= 0$, so the result is a non-negative number.

If you can formulate a problem as a positive semidefinite matrix, then you automatically constrain the result to the "same side." This constraint results in higher algorithmic efficiency.

Let's look at the formal definition:

A matrix is positive definite if, and only if:

$$x^T Ax > 0, x, x \neq 0$$

A matrix is positive semidefinite, if, and only if:

$$x^T Ax \geq 0$$

Further, a positive semidefinite matrix had an important property. Its eigenvalues are ≥ 0. The eigenvalues of a positive definite matrix are > 0.

Take the triplet (a, b, c) and the symmetric matrix:

$$\begin{bmatrix} 1 & a & b \\ a & 1 & c \\ b & c & 1 \end{bmatrix} \geq 0$$

For what values of a, b, c is this matrix positive semidefinite? If you iterate through all possible combinations of a,b,c, then compute the eigenvalues for each matrix, you will find that some (like $0, 0, 0$) result in a positive semidefinite matrix and some (like $2, 2, 2$) are not positive semidefinite. If you plot the set of triplets that result in a semidefinite matrix, you will see an elliptope. This shape guarantees an optimal solution.

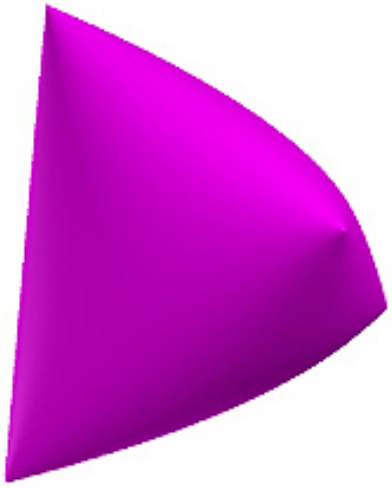


Figure 3.2: Elliptope visualized.

3.4 Identifying and Constructing a Positive Semidefinite Matrix

In the last section, you saw that being symmetric does not guarantee a positive semidefinite matrix. You also saw that a matrix of positive values does not guarantee a positive semidefinite matrix. The only way to check for positive semidefinite is to calculate the eigenvalues, which you learned in a previous workbook.

A surefire way to construct a positive semidefinite matrix is:

$$AA^T$$

Exercise 3 Figuring out if a matrix is positive semidefinite

Is this matrix positive definite? Show your work.

$$\begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix}$$

Working Space

Answer on Page 42

Exercise 4 Creating a positive semidefinite matrix

Using any 3 by 3 matrix, create a positive semidefinite matrix.Then show it is positive semidefinite by calculating its eigenvalues. You can either compute this by hand or using Python. In either case, show your work.

Working Space

Answer on Page 42

3.5 The Max Cut Problem

A famous NP problem is Max Cut. Given a graph of interconnected nodes, cut the graph to create two sets of nodes, such that the cut goes through as many edges as possible. (You can't cut an edge more than once.) Max Cut is important for binary classification in machine learning, circuit design, statistical physics, and more. There is no algorithm that will provide an exact solution. (If you could find one, you would be eligible to win a huge prize from the Clay Mathematics Institute!) Instead, you will see how to approximate a solution to this problem using a positive semidefinite matrix and a technique developed by mathematicians Michel Goemans and David Williamson.

You won't see all the complete details here, as this section is meant to be a quick introduction to how you can apply positive semidefinite matrices.

Take this simple graph of five nodes and six edges. Each node in the graph will take on

one of two values (1 or -1) to show which set they fall into after a cut is made. For any two connected nodes, $x_i \cdot x_j = 1$ if $x_i = x_j$ and -1 otherwise.

If you randomly assign 1 and -1 to the nodes, the chance of making the max cut is 0.5. By using semidefinite programming, you can achieve an algorithmic efficiency of 0.87.

The Goemans-Williamson technique can be used for any optimization problem where the variables take on the values of 1 and -1.

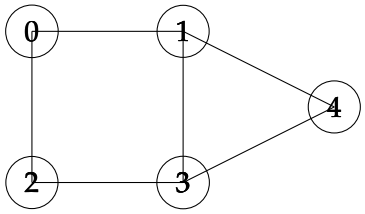


Figure 3.3: MaxCut algorithm visualized.

As a list of edges the graph is:

$$edges = [(0,1),(0,2),(1,3),(1,4),(2,3),(3,4)]$$

The optimization problem can be formulated as:

$$Max \sum_{edges(i,j)} \frac{1 - x_i x_j}{2}$$

for

$$x_i \in \{-1, 1\}$$

However, instead of allowing x_i to be scalar, Goemans-Williamson defines x_i as unit vectors.

$$x_i \in R^n, x_i = 1$$

and that makes the optimization equation:

$$Max \sum_{edges(i,j)} \frac{1 - x_i^T x_j}{2}$$

It is this "relaxation" that gets us to a semidefinite matrix, because we can now rewrite

the problem as a positive semidefinite matrix:

$$X = \left[x_i^T x_j\right]_{i,j}$$

Python has a module for solving optimization problems. Using this, you will get an optimum matrix, but to get the unit vectors, you will need to take the square root of the matrix.

$$X = [x_1 ... x_{1n}]\,[x_1 ... x_{1n}]^T$$

Next we need to go from unit vectors to scalars, using a process called rounding.

$$x^i \in R^n \rightarrow x^i \in \{-1, 1\}$$

Goemans-Williamson leveraged the fact that the end point of a unit vector is on a sphere. They generated a random plane to bisect the sphere. A vector on one side of the plane is assigned the value of 1 and a vector on the other side a value of -1.

You can then assign the scalar values to the nodes and make the cut accordingly. This particular cut will be 5, as shown.

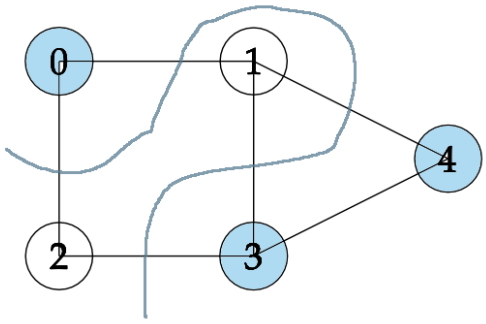


Figure 3.4: The result of the MaxCut.

3.6 The Max Cut Problem Solved in Python

```
# MaxCut Problem
import numpy as np
import scipy
from scipy.linalg import sqrtm
# cvxpy is a python module for solving optimization problems
import cvxpy as cp

# define the edges of the graph
edges = [(0,1),
         (0,2),
         (1,3),
         (1,4),
```

```
        (2,3),
        (3,4)]

#  Declare the matrix X to be positive semidefinite
X = cp.Variable((5,5),symmetric=True)
constraints = [X >> 0]

# Set diagonals to 1 to get unit vectors
constraints += [
    X[i,i] == 1 for i in range(5)
]

# Set the objective function
objective = sum(05.*(1 - X[i,j]) for (i,j)in edges)

# Set up the problem to maximize using the objective function and
# keeping within the set constraints
prob = cp.Problem(cp.Maximize(objective), constraints)

# Returns a positive semidefinite matrix
print(prob.solve())

# To get the vectors, take square root of the matrix
x = sqrtm(X.value)

# Generate a random hyperplane
u = np.random.randn(5) # normal to random hyperplane

# Pick values according to which side of the hyperplane the vectors are on
x = np.sign(x @ u)

```

CHAPTER 4

Introduction to C

You already know how to write programs in Python. In the next few chapters, you are going to learn C. Your Python knowledge will make this easier, because C has the same fundamental ideas: variables, functions, loops, and conditionals. The syntax looks different, and there are a few genuinely new concepts, but you will recognize the shape of everything you write.

Why learn C? Two reasons. First, C is used wherever performance matters: operating systems, game engines, embedded systems, and scientific computing. Second, and more important for this course, C forces you to think about memory explicitly. Python quietly manages memory on your behalf; C makes you do it yourself. Once you understand how C uses memory, you will also understand what Python is doing underneath, and you will be ready to implement data structures from scratch.

4.1 Compiled vs. Interpreted

When you run a Python program, the Python interpreter reads your `.py` file and executes it line by line. There is no separate step between writing and running.

C works differently. Before a C program can run, a *compiler* must translate it into machine code, the raw instructions that your CPU executes directly. The compiler reads your `.c` file, checks it for errors, and produces an *executable* file. You then run that executable.

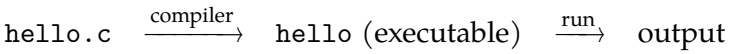


Figure 4.1: The compile-then-run cycle

Because the compiler sees your entire program before anything runs, it can catch many mistakes, like using a variable before declaring it or passing the wrong type to a function, that Python would only discover at runtime.

To compile a C file called `hello.c`, you run:

```
1 gcc hello.c -o hello
```

The `-o hello` flag names the output executable `hello`. Then you run it:

```
./hello
```

4.2 Your First C Program

Create a file called `hello.c` and type the following:

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

Compile and run it. You should see:

```
Hello, world!
```

Let's look at each piece.

`#include <stdio.h>` tells the compiler to include the standard input/output library, which provides `printf` for printing. Think of it like Python's `import` statement.

`int main()` is the function where your program starts. Every C program must have exactly one `main` function. The `int` means it returns an integer; `return 0` signals to the operating system that the program finished successfully.

`printf("Hello, world!\n");` prints a line of text. The `\n` is the newline character.

Notice the semicolons at the end of statements, and the curly braces `{ }` around the body of `main`. In Python, indentation defines blocks; in C, curly braces do.

4.3 Types and Variables

In Python, you write:

```python
x = 5
y = 3.14
```

Python figures out the type of each variable automatically.

In C, you must declare the type of every variable when you create it:

```c
int x = 5;
double y = 3.14;
```

This is called *static typing*. Once you declare a variable as `int`, it can only hold integers. The compiler enforces this and will refuse to compile code that mixes types incorrectly.

The common C types are:

`int` A whole number: `0`, `42`, `-7`. Typically 4 bytes.

`double` A floating-point number: `3.14`, `-0.001`. Typically 8 bytes. (There is also `float`, which uses 4 bytes and has less precision.)

`char` A single character: `'A'`, `'z'`, `'7'`. Use single quotes.

`int` Booleans in C are just integers: `0` means false, and any non-zero value means true. You can include `<stdbool.h>` to get the names `true` and `false`.

You can also declare a variable without immediately giving it a value:

```c
int count;      /* declared but not initialized */
count = 10;     /* now it has a value */
```

Warning: an uninitialized variable in C contains whatever random bytes happen to be in that memory location. Unlike Python, C will not raise an error if you read an uninitialized variable; it will just give you garbage. Always initialize your variables.

Note that C uses `/* ... */` for comments, not `#`. You can also use `//` for single-line comments in modern C.

4.4 Printing with `printf`

`printf` uses format specifiers to print different types:

```c
int age = 17;
double height = 1.75;
printf("Age: %d, Height: %f\n", age, height);
```

Output:

```
Age: 17, Height: 1.750000
```

Common format specifiers:

Specifier	Type
%d	int
%f	double or float
%c	char
%s	string (char pointer)

4.5 Conditionals and Loops

The logic is the same as Python; only the syntax changes.

4.5.1 if / else

```c
int score = 85;

if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else {
    printf("C or below\n");
}
```

The condition must be in parentheses `( )`, and each branch is surrounded by curly braces `{ }`.

4.5.2 for and while loops

```c
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

A C `for` loop has three parts separated by semicolons: the initializer (`int i = 0`), the

condition (`i < 5`), and the update (`i++`). The `++` increments by 1, the same as `i = i + 1`.

```c
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

4.6 Functions

In C, you must declare the return type and the type of each parameter:

```c
double square(double x)
{
    return x * x;
}

int main()
{
    printf("%f\n", square(4.0));   /* prints 16.000000 */
    printf("%f\n", square(2.5));   /* prints 6.250000  */
    return 0;
}
```

If a function returns nothing, its return type is `void`:

```c
void printGreeting(char *name)
{
    printf("Hello, %s!\n", name);
}
```

Functions must be defined *before* they are called, or you must provide a *forward declaration* (just the signature, ending in a semicolon) above `main`:

```c
double square(double x);   /* forward declaration */

int main()
{
    printf("%f\n", square(3.0));
    return 0;
}

```

```c
double square(double x)    /* full definition can come later */
{
    return x * x;
}
```

4.7 The Stack: Blackboards for Functions

Every function, while it is running, needs somewhere to store its local variables. C gives each function a *frame*: a reserved chunk of memory that holds all of that function's local variables and parameters.

Think of a frame as a **blackboard**. When a function starts running, it gets a fresh blackboard. It can write anything it wants on that blackboard, create variables, change their values. When the function finishes, the blackboard is *erased and discarded*. Local variables do not survive after their function returns.

Where do these frames live? In a region of memory called the *stack*. The stack works exactly like a physical stack of objects: the most recently added item is always on top. When `main` calls a function, that function's frame is pushed onto the top of the stack. When the function returns, its frame is popped off, and `main`'s frame is on top again.

Here is a concrete example:

```c
#include <stdio.h>

int cookingMinutes(int weightPounds)
{
    int minutes = 15 + 15 * weightPounds;
    return minutes;
}

int main()
{
    int totalWeight = 10;
    int gibletsWeight = 1;
    int turkeyWeight = totalWeight - gibletsWeight;
    int result = cookingMinutes(turkeyWeight);
    printf("Cook for %d minutes.\n", result);
    return 0;
}
```

When `main` calls `cookingMinutes(9)`, the stack looks like this:

cookingMinutes()	`weightPounds = 9` `minutes = 150`

main()	`totalWeight = 10`
	`gibletsWeight = 1`
	`turkeyWeight = 9`
	`result = ?`

`main`'s blackboard sits below; `cookingMinutes`'s blackboard sits on top of it. The variable `minutes` inside `cookingMinutes` is completely separate from anything in `main`. Each function only sees its own blackboard.

When `cookingMinutes` returns 150, its frame is discarded. `main`'s frame resumes, and `result` is filled in with 150.

4.7.1 Parameters are copies

When you call `cookingMinutes(turkeyWeight)`, C copies the value of `turkeyWeight` into the new parameter `weightPounds` on `cookingMinutes`'s blackboard. Changing `weightPounds` inside the function would not change `turkeyWeight` in `main`. They are separate variables. This is called *pass-by-value*.

4.8 Addresses and Pointers

Every variable in your program lives at some address in memory. You can find out the address of a variable using the `&` operator, called the *address-of* operator:

```c
#include <stdio.h>

int main()
{
    int i = 17;
    printf("i stores its value at %p\n", (void *)&i);
    return 0;
}
```

Run this and you will see something like:

```
i stores its value at 0x7ffeea3c4738
```

The address is printed in hexadecimal. Your address will differ, because the operating

system decides where in memory your variables live.

A *pointer* is a variable that holds an address. If you want to store the address of an `int`, you declare a variable of type `int *` ("pointer to int"):

```
int i = 17;
int *addressOfI = &i;
printf("i is at %p\n", (void *)addressOfI);
```

The `*` in the declaration is part of the type; it means "this variable holds an address, and the thing at that address is an `int`."

4.8.1 Reading and writing through a pointer

Once you have a pointer, you can use the `*` operator to read or write the value *at* the address the pointer holds. This is called *dereferencing* the pointer:

```
int i = 17;
int *p = &i;

printf("%d\n", *p);   /* prints 17 -- reads through p  */

*p = 89;              /* writes 89 to the address p holds */
printf("%d\n", i);    /* prints 89 -- i was changed!   */
```

To summarize:

- `&i` gives you the address of `i`.
- `int *p` declares a variable that can hold such an address.
- `*p` gives you the value stored at the address `p` holds.

4.8.2 NULL pointers

Sometimes you want a pointer that does not yet point to anything. Use `NULL`:

```
int *p = NULL;   /* p points to nothing */

if (p) {
    printf("%d\n", *p);     /* only runs if p is non-null */
```

```
} else {
    printf("p is null\n");
}
```

A null pointer is simply the address zero. **Never dereference a null pointer**, your program will crash. Always check before dereferencing a pointer that might be null.

4.9 Pass-by-Reference

Recall that when you call a function in C, arguments are *copied* into the function's frame. But what if you want a function to modify a variable in the calling function? You pass the *address* of that variable instead of its value. The function can then write to that address.

There is a standard C function called `modf()` that splits a floating-point number into its integer part and its fractional part, and needs to return *both*. Since a function can only return one value, it returns the fractional part and writes the integer part to an address you supply:

```
#include <stdio.h>
#include <math.h>

int main()
{
    double pi = 3.14;
    double integerPart;
    double fractionPart;

    /* Pass the address of integerPart so modf can write there */
    fractionPart = modf(pi, &integerPart);

    printf("Integer part:  %f\n", integerPart);
    printf("Fraction part: %f\n", fractionPart);
    return 0;
}
```

Output:

```
Integer part:  3.000000
Fraction part: 0.140000
```

Here is another way to think about it. Imagine you need to give a spy an assignment. You say: "I've left a length of steel pipe at the foot of the angel statue in the park. When you have the information, roll it up and leave it in the pipe. I'll pick it up Tuesday." In the spy

business, this is called a *dead drop*.

Pass-by-reference works the same way. You are not giving the function the data directly, you are giving it a *location* where it can leave the result.

Here is our own pass-by-reference function that converts meters to feet and inches:

```c
#include <stdio.h>
#include <math.h>

void metersToFeetAndInches(double meters,
                           int *ftPtr,
                           double *inPtr)
{
    double rawFeet = meters * 3.281;
    int feet = (int)floor(rawFeet);
    double inches = (rawFeet - feet) * 12.0;

    if (ftPtr) *ftPtr = feet;
    if (inPtr) *inPtr = inches;
}

int main()
{
    int feet;
    double inches;
    metersToFeetAndInches(1.8, &feet, &inches);
    printf("%d ft %f in\n", feet, inches);
    return 0;
}
```

The caller declares the variables it wants filled in (`feet`, `inches`), passes their addresses to the function, and the function writes the results there.

4.10 Structs: Grouping Related Data

A *struct* lets you bundle several related values into a single named type. In Python you might use a dictionary for this; in C, a struct is the fundamental grouping mechanism.

```c
struct Person {
    double heightInMeters;
    int weightInKilos;
};
```

This defines a new type called `struct Person`. You can then create variables of that type:

```c
#include <stdio.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person p)
{
    return p.weightInKilos / (p.heightInMeters * p.heightInMeters);
}

int main()
{
    struct Person mikey;
    mikey.heightInMeters = 1.70;
    mikey.weightInKilos  = 96;

    struct Person aaron;
    aaron.heightInMeters = 1.97;
    aaron.weightInKilos  = 84;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));
    printf("aaron BMI: %f\n", bodyMassIndex(aaron));
    return 0;
}
```

You access the members of a struct using a dot: `mikey.heightInMeters`.

When `main` is running, the stack looks like this:

main()	`mikey.heightInMeters = 1.70`
	`mikey.weightInKilos = 96`
	`aaron.heightInMeters = 1.97`
	`aaron.weightInKilos = 84`

The two `Person` structs live right inside `main`'s frame on the stack.

4.11 The Heap

The stack is fast and automatic, but it has two constraints: stack frames are destroyed when functions return, and the sizes of all stack-allocated variables must be known at compile time.

The *heap* is a separate region of memory that has neither constraint. Memory on the heap

persists until you explicitly release it, and can be any size determined at runtime.

In C, you claim heap memory with `malloc` and release it with `free`.

```c
#include <stdlib.h>

struct Person *mikey = malloc(sizeof(struct Person));
```

`malloc` asks the system for enough heap memory to hold a `Person` struct and returns the address of that memory. The variable `mikey` is a *pointer* to a `Person`, it lives on the stack, but the struct it points to lives on the heap.

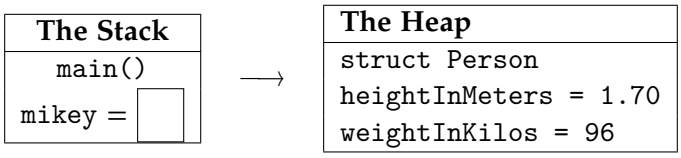


Figure 4.2: A pointer on the stack pointing to a struct on the heap

4.11.1 The arrow operator

When you have a pointer to a struct, you access its members using `->` instead of a dot:

```c
mikey->heightInMeters = 1.70;
mikey->weightInKilos  = 96;
```

The code `mikey->weightInKilos` means: "dereference the pointer `mikey` to get the struct, then access its `weightInKilos` member." It is shorthand for `(*mikey).weightInKilos`.

4.11.2 Releasing heap memory

When you are done with heap-allocated memory, you must release it with `free`:

```c
free(mikey);
mikey = NULL;   /* good habit: prevents accidental reuse */
```

After `free`, the memory is returned to the heap for reuse. Setting the pointer to `NULL` prevents you from accidentally using it again.

If you forget to call `free`, the memory is never returned. This is called a *memory leak*. In a short-lived program it may not matter, but in a long-running program it can consume all available memory.

4.11.3 A complete heap example

```c
#include <stdio.h>
#include <stdlib.h>

struct Person {
    double heightInMeters;
    int weightInKilos;
};

double bodyMassIndex(struct Person *p)
{
    return p->weightInKilos / (p->heightInMeters * p->heightInMeters);
}

int main()
{
    struct Person *mikey = malloc(sizeof(struct Person));
    mikey->heightInMeters = 1.70;
    mikey->weightInKilos  = 96;

    printf("mikey BMI: %f\n", bodyMassIndex(mikey));

    free(mikey);
    mikey = NULL;

    return 0;
}
```

`bodyMassIndex` now takes a `struct Person *` instead of a `struct Person` by value. This avoids copying the struct. Functions that work on heap-allocated data almost always take pointers.

4.11.4 Heap arrays

You can also allocate a contiguous block of memory for multiple values. Use `malloc` with a count:

```c
/* Allocate space for 1000 doubles on the heap */
double *buffer = malloc(1000 * sizeof(double));
```

```
3
4  buffer[0] = 3.14;
5  buffer[1] = 2.72;
6
7  free(buffer);
8  buffer = NULL;
```

A pointer to the first element is all you need to work with the whole array. The heap memory is contiguous, so `buffer[0]`, `buffer[1]`, and so on are laid out one after another.

4.12 What Python Objects Actually Are

You now know everything you need to understand what a Python object is, under the hood.

When Python creates an object, it:

1. Allocates a struct on the heap large enough to hold the object's data (its instance variables).
2. Stores in that struct a pointer to the class's method table, so that method calls can find the right code.
3. Returns a pointer to that struct.

Your Python variable `obj` is, at the machine level, a pointer to a heap-allocated struct. When Python's garbage collector determines that nothing points to an object anymore, it calls the equivalent of `free` to release that memory.

The next chapter introduces C++ classes, and with this memory model in hand, you will implement linked lists, trees, and hash tables from scratch.

4.13 Exercises

1. Write a C function `celsiusToFahrenheit` that takes a `double` temperature in Celsius and returns the equivalent in Fahrenheit. Call it from `main` for the values 0, 20, 37, and 100, and print each result.
2. Write a function `swap` that takes two `int` pointers and swaps the values at the two addresses. Test it so that after calling `swap(&a, &b)`, the values of `a` and `b` have been exchanged.
3. Write a struct `Point` with `double` members `x` and `y`. Write a function `distance` that takes two `struct Point *` pointers and returns the Euclidean distance between

them. Allocate two points on the heap, fill in their coordinates, print the distance, and then free the memory.

4. **Memory leak hunt.** The following program has a memory leak. Identify and fix it.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;
};

void printValue(int x)
{
    struct Node *n = malloc(sizeof(struct Node));
    n->value = x;
    printf("%d\n", n->value);
}

int main()
{
    for (int i = 0; i < 5; i++) {
        printValue(i);
    }
    return 0;
}
```

5. A `struct tm` is defined in `<time.h>` and holds a broken-down calendar time. The function `time(NULL)` returns the number of seconds since midnight, January 1, 1970. The function `localtime_r(&seconds, &now)` fills a `struct tm` with the corresponding date and time. Here is a snippet to get you started:

```c
#include <stdio.h>
#include <time.h>

int main()
{
    long secondsSince1970 = time(NULL);
    struct tm now;
    localtime_r(&secondsSince1970, &now);
    printf("The time is %d:%d:%d\n",
           now.tm_hour, now.tm_min, now.tm_sec);
    return 0;
}
```

Look up the other fields of `struct tm` and modify the program to print today's full date. Then modify it to print the date that falls exactly 4,000,000 seconds from now. (Hint: `tm_mon` is 0-indexed, so January is 0.)

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 6)

The first vector of the orthogonal subspace is:

$$v_1 = x_1 = (1, 1, 1)$$

The second vector of the subspace is a projection of x_2 onto v_1.

$$v_2 = x_2 - \frac{x_2 v_1}{v_1 v_1} v_1$$

Substitute the values:

$$v_2 = (0, 1, 1) - \frac{(0, 1, 1)(1, 1, 1)}{(1, 1, 1)(1, 1, -1)}(1, 1, 1)$$

$$v_2 = (0, 1, 1) - (2/3)(1, 1, 1)$$

$$v_2 = (-2/3, 1/3, 1/3)$$

Normalize:

$$v_1 = v_1 / \sqrt{|v_1|}$$

$$v_1 = (1, 1, 1) / \sqrt{|v_1|}$$

$$v_1 = (0.577, 0.577, 0.577)$$

$$v_2 = v_2 / \sqrt{|v_2|}$$

$$v_2 = (0, 1, 1) \sqrt{|v_2|}$$

$$v_2 = (-0.816, 0.408, 0.408)$$

Answer to Exercise 2 (on page 14)

$$U = \begin{bmatrix} -0.70710678 & -0.70710678 \\ -0.70710678 & 0.70710678 \end{bmatrix}$$

$$Singular values = [3.464101623.16227766]$$

$$V^T = \begin{bmatrix} -0.408 & -0.816 & -0.408 \\ -0.894 & 0.447 & 0.0 \\ -0.183 & -0.365 & 0.9129 \end{bmatrix}$$

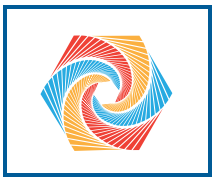
Answer to Exercise 3 (on page 21)

Yes. Its eigenvalues are

$$2, 2$$

Answer to Exercise 4 (on page 21)

The answer depends on the 3 by 3 matrix you chose.



INDEX

address-of operator, 31
arrow operator, 36

C types, 27
compiler, 25

dereferencing, 32

free, 36

Gram-Schmidt Process, 3
Gram-Schmidt process
    python script for, 6

heap, 35
    arrays, 37

malloc, 36
Max Cut, 21
    python script for , 23
memory leak, 37
modf, 33

NULL, 32

pass-by-reference, 33
pass-by-value, 31
pointer, 31, 32

singular value decomposition, 9
stack, 30
stack frame, 30
static typing, 27
struct, 34
svd, 9
    python script for, 13